Aurélie SITJA L3
à l'Institut Villebon - *Georges Charpak*

2026

Rapport de stage de L3 : Nombres Pseudo-Aléatoires (PRNG)

Stage effectué à l'Institut de Mathématique d'Orsay,
pendant une durée de 8 semaines, du 4 mai au 19 juin 2026

Maître de stage :
Joël COHEN

Enseignant référent :
Cyril DAUPHIN

université PARIS-SACLAY
Institut Villebon Georges Charpak
Laboratoire de Mathématiques d'Orsay

Remerciements

Je remercie Joël COHEN d'avoir bien voulu me prendre et de nous avoir supervisé Jules et moi.
Je remercie l'Institut de Mathématique d'Orsay, l'organisme d'accueil, de m'avoir accueillie.
Et enfin, je remercie l'Institut Villebon - *Georges Charpak* de m'avoir donné l'occasion de faire ce stage qui m'a permis d'imaginer la vie future.

Résumé

Ce rapport présente le travail réalisé lors d'un stage effectué à l'Institut de Mathématique d'Orsay, bâtiment 307. Le sujet du stage portait sur les générateurs de nombres pseudo-aléatoires (Pseudo-Random Number Generators ou PRNG), des algorithmes permettant de produire des suites numériques reproduisant certaines propriétés du hasard tout en reposant sur des mécanismes déterministes.

Après une présentation de l'histoire et de l'évolution des PRNG, ce rapport introduit les principales notions mathématiques utilisées pour évaluer leur qualité, telles que la période, l'équirépartition, les k-uplets ou encore les corrélations statistiques. Plusieurs familles de générateurs sont ensuite étudiées, notamment les générateurs congruentiels linéaires, le Mersenne Twister et le générateur cryptographique Blum Blum Shub.

Une partie importante du stage a consisté à implémenter et analyser différents générateurs, ainsi qu'à étudier certaines méthodes permettant de retrouver leurs paramètres ou de prédire leurs sorties. Un travail particulier a également été réalisé autour des suites de de Bruijn et de leur génération à l'aide d'outils informatiques.

Ce projet m'a permis de porter un regard différent sur l'aléatoire. Derrière des suites de nombres qui semblent imprévisibles se cachent souvent des structures mathématiques riches, qu'il est possible d'étudier, de reproduire et parfois même d'exploiter pour retrouver le fonctionnement du générateur.

Sommaire

Sommaire des figures et images..... p2
Liste des abréviations..... p3
I / Introduction..... p4
II / Histoire et évolution des PRNG..... p4
a. Origine des PRNG..... p4
b. Premiers générateurs..... p5
c. Évolution moderne..... p5
III / Fondements théoriques des PRNG..... p6
a. État interne, graine et période..... p6
b. Équirépartition et k-uplets..... p6
c. Corrélations et défauts statistiques..... p9
IV / Implémentation et attaques..... p9
a. Générateur Congruentiel Linéaire..... p9
b. Suite de de Bruijn..... p10
V / Conclusion et Perspectives..... p12
VI / Activité de médiation..... p13
VII / Références..... p15

Sommaire des figures et images

Image 1 : Échantillon de chiffres décimaux aléatoires
Image 2 : Tracé tridimensionnel de 100 000 valeurs de triplets générés avec RANDU
Image 3 : Photo de l'activité tenue au stand de Math en Scène et Compagnie

Liste des abréviations

PRNG : nombres pseudo-aléatoires / génération de nombres pseudo-aléatoires
MT : Mersenne Twister
BBS : Blum Blum Shub
LCG : Générateur Congruentiel Linéaire
PCG : Générateur Congruentiel Permuté

I / Introduction

De nos jours, les nombres aléatoires sont utilisés dans de nombreux domaines tels que les simulations scientifiques, les statistiques, les jeux vidéo, l'intelligence artificielle ou encore la cybersécurité. Dans la plupart des cas, ces nombres sont produits par des générateurs de nombres pseudo-aléatoires (PRNG). Bien que leurs sorties paraissent aléatoires, ces générateurs reposent en réalité sur des algorithmes déterministes et des propriétés mathématiques précises.

Mon stage porte sur les aspects mathématiques (construction, propriétés statistiques, limites) et informatiques (implémentation, détection, prédiction). Il s'est déroulé dans le campus de l'université Paris-Saclay, Institut de Mathématique d'Orsay (bâtiment 307).

II / Histoire et évolution des PRNG

a. Origine des PRNG

L'idée des générateurs de nombres pseudo-aléatoires (PRNG) est apparue bien avant l'apparition des ordinateurs, bien que leur nom n'était pas le même. À l'époque, il fallait lancer des dés ou jouer aux cartes à la main pour avoir des statistiques. C'est pourquoi avec ce besoin de hasard à automatiser, les tables de nombres aléatoires ont vu le jour. La première table de ce type a été publiée par LHC Tippett, un statisticien, en 1927. Le but de ces tables est de posséder toutes les propriétés du hasard, c'est-à-dire, que les nombres soient choisis en fonction des colonnes, des lignes ou aléatoirement, les propriétés seront respectées.

01581	36001	15892	57621	85239
02972	56177	87580	66794	48123
97022	65380	91304	32853	99729
67583	01277	77815	60558	75920
26935	56306	38710	77239	47139
21201	75983	35695	60517	14579
02628	26124	68322	01436	85994
93635	69404	76323	33459	70041
08984	81320	03226	60959	78246
04415	78662	28295	46513	92889
13070	18401	14382	48262	53177
53531	36891	29620	72532	47368
87733	74995	61843	88472	15736
47619	57452	92819	34401	48782
94060	67951	28895	79309	91897

Image 1 : Échantillon de chiffres décimaux aléatoires

Dans les années 1940, le développement des méthodes de Monte-Carlo, qui consiste à résoudre un problème compliqué en simulant énormément d'événements aléatoires, a fortement augmenté les besoins en nombres aléatoires. Les tables de nombres aléatoires utilisées jusque-là étant peu adaptées aux calculs massifs. Les PRNG sont alors apparus comme une solution permettant de produire rapidement de longues suites de nombres ayant des propriétés proches de celles du hasard.

b. Premiers générateurs

Le premier générateur de PRGN a été la méthode du carré médian, décrite par Von Neumann en 1949. Celle-ci se résumait à prendre un nombre à x chiffres, de l'élever au carré, puis de prendre les x chiffres du milieu. Bien qu'il n'est pas parfait dû aux séquences de destruction présente (nombre avec x 0 sur le milieu), ce facteur est en sa faveur, car il est facilement détectable.

En 1960 un autre générateur a été introduit, RANDU, un générateur congruentiel linéaire (LCG) de forme $X_{n+1} = 65539 \times X_n \left[2^{31}\right]$ et génère un réel entre 0 et 1 : $r_n = \frac{X_n}{2^{31}}$. Ici, l'écriture $x[m]$ signifie reste de x dans la division entière par m. Cependant malgré son fort potentiel il possède de nombreuses lacunes telles que ses importantes corrélations qu'on peut visualiser sur un plan en trois dimensions. Étant bien ancré grâce à sa compatibilité et n'ayant pas de remplaçant supérieur, il fut utilisé jusqu'au année 1970-80.

c. Évolution moderne

Parmi les évolutions modernes on peut y noter le Mersenne Twister (MT), codé en bit et ne possédant que des 0 et des 1. Basé sur les nombres de Mersenne (nombre premier de forme $2^n - 1$) sa période est de $2^{19937} - 1$ et utilise une implémentation de 32 bits. En revanche, il n'est pas utilisé en cryptographie, car il possède un comportement trop prédictible. C'est pourquoi un autre algorithme est implémenté, le Blum Blum Shub (BBS).

L'algorithme BBS est défini par $X_{n+1} = X_n^{\,2}\,[m]$, la condition de son fonctionnement est que X_0 doit être premier avec m. Par exemple, si $m = 11 \times 23 = 253$, et $X_0 = 3$, alors $X_1 = 3^2\,[253] = 9$, $X_2 = 9^2\,[253] = 81$, $X_3 = 81^2\,[253] = 236$, on trouve la suite :

3, 9, 81, 236, 36, 31, 202, 71, 234, 108, 26, 170, 58, 75, 59, 192, 179, 163, …

Contrairement au MT, BBS a été conçu avec des objectifs cryptographiques.

Il y aussi le Générateur Congruentiel Permuté (PCG), un dérivé du LCG, qui produit des nombres binaires équirépartis, mais permutés.

III / Fondements théoriques des PRNG

a. État interne, graine et période

Avant d'étudier des générateurs particuliers, il est nécessaire de présenter quelques notions communes à l'ensemble des PRNG.

Un générateur pseudo-aléatoire est un algorithme déterministe produisant une suite de nombres qui cherche à imiter certaines propriétés du hasard. Chaque PRNG possède un état interne, généralement initialisé par une valeur appelée graine (seed). À partir de cet état, le générateur calcule un nouvel état puis produit une sortie. Deux exécutions utilisant la même graine produiront exactement la même suite de nombres.

Une conséquence directe de ce caractère déterministe est l'existence d'une période. Comme le nombre d'états possibles est fini, un état déjà rencontré finira nécessairement par réapparaître. À partir de ce moment, la suite recommence à l'identique. La longueur de la période constitue donc un critère important pour évaluer la qualité d'un générateur.

Cependant, une grande période n'est pas suffisante. Un bon PRNG doit également produire des valeurs uniformément réparties et limiter les corrélations entre termes successifs. Cette propriété est souvent étudiée à travers l'analyse des k-uplets, c'est-à-dire des groupes de k valeurs consécutives. Un générateur peut produire des valeurs individuellement bien réparties tout en présentant des motifs ou des structures détectables lorsque plusieurs valeurs sont observées simultanément.

b. Équirépartition et k-uplets

Nous pouvons maintenant aborder la partie la plus importante de l'aléatoire déterministe, les k-uplets. Comme évoqué plus tôt, un bon PRNG doit limiter au maximum les corrélations entre termes successifs. Plus il est nécessaire de connaître de termes pour prédire la suite, mieux c'est, au contraire si 2 ou 3 suffisent le générateur est très mauvais.

Prenons E ensemble fini de valeur (exemple $E = \{0, 1\}$) et $\left(u_n\right)_{n\geq 0}$ une suite dans E.

Def k-équirépartition :

Soit $k \in \mathbb{N}^*$, on dit que $\left(u_n\right)_{n\geq 0}$ est :

① k-équirépartie si $\forall \left(e_1, e_2, \dots, e_k\right) \in E^k$,

$$\lim_{n \to +\infty} \frac{\left|\left\{ i \leq n \mid \left(e_i, e_{i+1}, \dots, e_{i+k}\right) = \left(e_1, e_2, \dots, e_k\right) \right\}\right|}{n-k} = \frac{1}{\left|E^k\right|}$$

② k-équirépartie si $\forall\, k \in \mathbb{N}^*$ chaque k-uplet apparaît et à même fréquence d'apparition

En particulier :

- pour $k = 1$, cela signifie que toutes les valeurs de E sont prises avec la même fréquence.
- pour $k = 2$ tous les couples de valeur apparaissent à même fréquence, cela revient à dire que u_n et u_{n+1} sont indépendants.
- pour $k = i$, cela revient à l'indépendance de u_n , u_{n+1} , … , u_{n+i} .

Remarque :

- La k-équirépartition entraîne les k'-équirépartitions pour tout $k' \leq k$
- En pratique, si la suite $\left(u_n\right)$ représente les valeurs générées par un générateur aléatoire, on n'aura accès qu'à un échantillon fini de N valeurs et donc on ne pourra évaluer la k-équipartition que jusqu'à $k = log_{|E|}(N)$.
- Par la suite dans ce document on abandonnera l'objectif de l'équirépartition pour tout k (possible théoriquement, mais en pratique très difficile), et on se contentera de demander l'équirépartition jusqu'à une certaine valeur de k adaptée à la taille typique des échantillons de valeurs qu'on va générer avec notre algorithme.
- Par extension (ou abus de langage), on dira qu'une suite qui parcourt tous les k-uplets sauf 1 est encore de de Bruijn.

Exemple, la suite 010101… est 1-équirépartie, mais clairement pas 2-équirépartie (par exemple le couple 00 n'y apparaît jamais). La conséquence pratique est que la suite devient prévisible dès qu'on en connaît un seul terme. Après un 0 vient un 1 et inversement. Corsons maintenant la chose avec cette suite 00110011… . D'un seul coup connaître un chiffre ne sert à rien, après un 0 peut aussi bien suivre un autre 0 qu'un 1. En revanche si on en connaît deux le reste se sait aussi.

Une façon simple de garantir la k-équirépartition d'une suite serait de parcourir de manière périodique tous les k-uplets dans un certain ordre (à supposer que ça soit possible). Par exemple, la suite 8-périodique 00010111 00010111… parcourt tous les triplets de 0,1 possible. Elle est donc 3-équipartie.

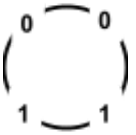
Ce principe est utilisé par les suites de de Bruijn et par le Mersenne Twister.

Def :

Une suite (u_n) est dite de de Bruijn d'ordre k à valeur dans E si elle est $|E^k|$-périodique et parcourt une et une seule fois tout les k-uplets de E^k sur la période :

$$\forall\, (e_1, e_2, \dots, e_k) \in E^k,\ \exists!\ n_0 \leq |E^k|,\ (u_{n_0}, u_{n_0+1},\ \dots,\ u_{n_0+k}) = (e_1, e_2,\ \dots,\ e_k)$$

Par exemple la suite 4-périodique 00110011… est de de Bruijn d'ordre 2 (à valeurs dans $\{0, 1\}$) car apparaissent successivement dans la période les couples 00, 01, 11 et 10. On peut l'illustrer en représentant la période de manière cyclique. En conséquence cette suite est 2-équirépartie (mais pas 3-équirépartie : par exemple le triplet 000 n'apparaît jamais).



La suite 8-périodique 00010111 00010111… est de de Bruijn d'ordre 3 (à valeurs dans $\{0, 1\}$). En effet, on y voit apparaître successivement les triplets 000, 001, 010, 011, 111, 110 et 100. Donc cette suite est 3-équirépartie.

La suite 16-périodique 0000100110101111 0000100110101111… est de de Bruijn d'ordre 4 (à valeurs dans $\{0, 1\}$).

La suite 9-périodique 001120221 001120221… est de de Bruijn d'ordre 2 (à valeurs dans $\{0, 1, 2\}$) car on y voit successivement les couples 00, 01, 11, 12, 20, 02, 22, 21, 10.

L'intérêt de cette notion est que de telles suites existent toujours ! En effet, on peut démontrer que pour tout k et tout ensemble fini E, il existe une suite de de Bruijn d'ordre k à valeur dans E (il en existe même souvent beaucoup). Et en plus, si $|E|$ est premier et pour certaines valeurs particulières de k, on sait même fabriquer de telles suites à partir d'une récurrence simple (ce qui sera utile pour fabriquer des PRNG et qui constitue la base de l'algorithme Mersenne Twister).

c. Corrélations et défauts statistiques

Les critères présentés précédemment permettent d'évaluer la qualité statistique d'un générateur. Cependant, même lorsqu'un PRNG possède une longue période et de bonnes propriétés d'équirépartition, des structures mathématiques peuvent subsister dans les suites produites. Ces structures ne sont pas toujours visibles directement. Elles apparaissent souvent lors de représentations géométriques, d'analyses spectrales ou à l'aide de tests statistiques spécialisés. Leur présence révèle que certaines valeurs successives ne sont pas totalement indépendantes.

L'exemple historique le plus connu est RANDU. Bien qu'il ait été largement utilisé pendant plusieurs années, l'étude de ses triplets successifs a montré que ceux-ci se répartissaient sur un nombre limité de plans dans l'espace.
Cette propriété traduit l'existence de corrélations internes et explique pourquoi RANDU est aujourd'hui considéré comme inadapté aux simulations scientifiques.

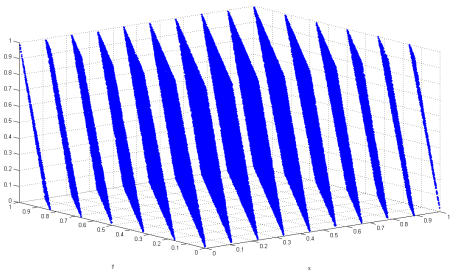


Image 2 : Tracé tridimensionnel de 100 000 valeurs de triplets générés avec RANDU

L'existence de telles corrélations est également à l'origine de nombreuses attaques. Lorsqu'un générateur laisse apparaître suffisamment d'information sur son état interne, l'observation d'un nombre limité de sorties peut permettre de retrouver certains paramètres du modèle ou de prédire les valeurs futures. L'étude de ces défauts constitue donc un aspect important de l'analyse des PRNG.

IV / Implémentation et attaques

a. Générateur Congruentiel Linéaire

La première reproduction est un modèle proche de celui de RANDU nommé Générateur Congruentiel Linéaire. De forme $X_{n+1} = \left(aX_n + b\right)[m]$, avec m premier, c'est l'un des générateurs les plus faciles à créer. Il renvoie une liste de nombres aléatoires de période maximale m.

Différentes attaques sont possibles, soit en connaissant m, soit en employant d'autres méthodes basées sur des suppositions. Prenons le premier cas, celui où on connaît m. Étant donné m premier il suffit de résoudre un système linéaire pour trouver a et b, X_0 étant tout simplement le premier terme sorti.

$$\begin{cases} X_1 = \left(a \times X_0 + b\right)[m] \\ X_2 = \left(a \times X_1 + b\right)[m] \end{cases}$$

Une fois a et b trouvé on peut recréer entièrement la suite.

Pour le deuxième cas, tout d'abord on sait que pour avoir une bonne suite m est premier, on part donc de ce cas de figure. Aussi la suite renvoyée nous donne de précieuses informations comme la période ou le nombre maximal obtenu, de là on peut en déduire un ordre de grandeur du nombre premier m utilisé, il ne reste alors qu'à chercher parmi les possibilités restantes.

Cette méthode nécessite un plus grand nombre de valeurs si on veut trouver a, b et m plus rapidement.

La période nous apporte un ordre de grandeur minimal de m en plus de borner la suite $T \leq m$. Pas forcément nécessaire mais utile quand il y a un grand nombre de valeurs. Après cela on peut chercher X_k le maximum entre X_0 et X_T. X_k est forcément inférieur à m.

À partir de là on augmente la valeur de m jusqu'à ce qu'il soit premier. On créer une nouvelle suite à partir de cette valeur estimée de m en utilisant la méthode pour trouver a et b, puis on la compare avec les valeurs réelles. Si les valeurs ne correspondent pas on recommence jusqu'à trouver le prochain nombre premier.

b. Suite de de Bruijn

Pour ce qui est des suites de de Bruijn, celle réalisée est d'ordre 5. Elle est définie par $u_{n+5} = u_n + u_{n+2}\,[2]$ (à valeurs dans $\{0, 1\}$).

Expliquons brièvement l'idée qui permet de construire une suite de de Bruijn récurrente.

Remarquons d'abord que fabriquer une suite récurrente linéaire d'ordre k revient à fabriquer une suite de k-uplets récurrente linéaire d'ordre 1 : en effet, la suite $(u_n)_{n \geq 0}$ vérifie la relation de récurrence $u_{n+k} = \sum_{i=0}^{k-1} \lambda_i u_{n+i}$ si et seulement si la suite $X_n = (u_n, u_{n+1}, \dots u_{n+k-1})$ des k-uplets vérifie la relation de récurrence $X_{n+1} = A X_n$ avec A la matrice compagnon :

$$A = \begin{bmatrix} 0 & 1 & 0 & . & 0 \\ 0 & 0 & 1 & & \\ . & & & . & \\ . & & & & 1 \\ \lambda_1 & \lambda_2 & . & . & \lambda_n \end{bmatrix}$$

On va donc chercher à fabriquer la suite (X_n). Dans ce formalisme, la suite $(u_n)_{n \geq 0}$ est de de Bruijn si et seulement si (X_n) prend $T = |E|^k$ valeurs toutes distinctes, et cela revient exactement à ce qu'elle soit T-périodique (car sa période correspond exactement à l'écart entre deux répétitions puisqu'elle est récurrente). Mais comme $X_n = A^n X_0$, il faut choisir la matrice A d'ordre T (c'est-à-dire telle que $A^T = I_n$ et $\forall 0 < k < T, A^k \neq I_n$). On utilise SageMath pour trouver une telle matrice : on sait que $\chi_A = X^k - \sum_{i=0}^{k-1} \lambda_i X^i$ est annulateur de A, et on souhaite que $X^T - 1$ annule A, il faut choisir χ_A parmi les diviseurs du polynôme $X^T - 1$. Une fois χ_A connu, on en déduit A et la suite (X_n).

Pour une suite de de Bruijn d'ordre 5 la matrice est donc :

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ a & b & c & d & e \end{bmatrix}$$

Nous avons $N = 2^5 - 1 = 31$.

$$X^{31} - 1 = (X + 1)\left(X^5 + X^2 + 1\right)\left(X^5 + X^3 + 1\right)\left(X^5 + X^3 + X^2 + X + 1\right)$$
$$\left(X^5 + X^4 + X^2 + X + 1\right)\left(X^5 + X^4 + X^3 + X + 1\right)$$
$$\left(X^5 + X^4 + X^3 + X^2 + X + 1\right)$$

L'une des factorisations possible du polynôme est : $\chi_A = X^5 + X^2 + 1$, ce qui donne :

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix}$$

Et donc : $u_{n+5} = u_n + u_{n+2}$ [2]. Mais suivant cette formule un uplet n'apparaîtra jamais, le 00000. C'est pourquoi lorsque le uplet 10000 apparaît le programme ajoute 01 (1000001), puis continue normalement.

On aurait aussi pu prendre : $\chi_A = X^5 + X^3 + 1$, ce qui donne $u_{n+5} = u_n + u_{n+3}$ [2] :

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

V / Conclusion et Perspectives

Ce stage m'a permis de me rendre compte que l'aléatoire n'est pas aussi aléatoire qu'on pourrait le penser, car possédant certaines lois on peut essayer de le simuler et de le recréer sur des plateformes déterministes. L'un des principaux enseignements de ce stage est qu'un générateur de nombres pseudo-aléatoires ne cherche pas à reproduire le hasard parfait, mais uniquement certaines de ses propriétés. La qualité d'un PRNG ne dépend donc pas uniquement de sa période ou de l'apparence de ses sorties, mais également de propriétés plus subtiles telles que l'équirépartition ou l'absence de corrélations détectables.

Pour pousser plus loin ce stage, une poursuite naturelle consisterait à étudier les méthodes d'attaque applicables aux suites de de Bruijn. Il serait également intéressant de généraliser l'implémentation réalisée afin de permettre la génération de suites d'ordre variable, plutôt que de se limiter aux 5-uplets. Cette généralisation n'a pas pu être réalisée dans le cadre du stage, le programme SageMath utilisé pour les expérimentations ayant été développé indépendamment du code Python présenté dans ce rapport et n'étant donc pas disponible sur le dépôt GitHub associé.

VI / Activité de médiation

Du 21 au 24 mai se tenait le Salon Culture et Jeux Mathématiques, dans lequel nous avons animé une activité au stand Math en Scène et Compagnie.

“ Le Salon Culture & Jeux Mathématiques se veut une grande fête des mathématiques, une manifestation originale et unique qui souhaite moderniser l’image des mathématiques, prouver que ces dernières sont universelles, présentes partout, innovantes, fascinantes et étonnantes. ”

Notre activité se résumait à présenter un tour de magie et d’expliquer son principe mathématique et sa réalisation. Plus précisément, il consiste à avoir un jeu préparé, trier dans un ordre voulu, puis d’effectuer deux riffles shuffles suivie d’une coupe (ou non). Après quoi on laisse le spectateur piocher la carte au sommet et la réinsérer dans le paquet au milieu. Il peut aussi effectuer une coupe avant de rendre le paquet. Le tour de magie utilise le principe des rising sequences (séquences de cartes qui se suivent dans le sens/ordre du paquets initial/état 0).

État 0 :
- cartes classées dans un ordre précis, 1 rising sequence

État 2 :
- un riffle shuffle effectué, 2 rising sequences

État 3 :
- un deuxième riffle shuffle effectué, 4 rising sequences

État 4 :
- une carte déplacée et insérée à un nouvel endroit, 5 rising sequences

C’est grâce à ce principe que la carte déplacée et vue par le spectateur est trouvée, bien qu’il peut échouer si la carte est malheureusement réinsérée dans une autre rising sequence. Aussi la coupe n’affecte pas cet ordre, c’est une illusion de contrôle donnée au spectateur, celle-ci transforme les rising sequences en rising sequences cyclique.

Exemple : mon paquet est 0123456789, j'effectue un riffle shuffle et j'obtiens 0152637894. Ce qui donne 2 rising sequences, respectivement 01234 et 56789. Si le 4 est déplacé en premier, deuxième ou troisième position il y aura toujours 2 rising sequences au lieu des 3 attendues.

0415263789 ce qui donne 0123 et 456789, soit aucun moyen de retrouver la carte hormis par déduction astucieuse.

L'activité a été tenue le vendredi par Jules et moi, et le samedi seulement par moi. J'ai trouvé la tenue du stand plutôt amusante, avec une préférence pour le samedi car le public était plus averti, aussi le vendredi était avec les scolaires et surtout sur rendez-vous. Faire le tour de magie et voir le public se demander "comment ?" puis y répondre a été une bonne expérience.



Image 3 : Photo de l'activité tenue au stand de Math en Scène et Compagnie

VII / Références

1. Sitja, A. (s.d.). Stage_L3_PRNG [en ligne], https://github.com/sleepylirea/Stage_L3_PRNG.git.
2. Schutzwerk GmbH. (2021). Attacking a Random Number Generator [en ligne], https://www.schutzwerk.com/blog/attacking-a-rng/.
3. Kelsey, J. et al. (1998). Cryptanalytic Attacks on Pseudorandom Number Generators. In Fast Software Encryption, Lecture Notes in Computer Science, vol. 1372, p. 168-188.
4. Schneier, B. (1994). Pseudo-Random Sequence Generators and Cryptographic Applications. Cryptologia, vol. 18 (2), p. 111-134 [en ligne], https://www.schneier.com/wp-content/uploads/2017/10/paper-prngs.pdf.
5. Blum, L. et al. (1986). A Simple Unpredictable Pseudo-Random Number Generator. SIAM Journal on Computing, vol. 15 (2), p. 364-383.
6. de Bruijn, N. G. (1946). A Combinatorial Problem. Koninklijke Nederlandse Akademie van Wetenschappen, vol. 49, p. 758-764 [en ligne], https://alexandria.tue.nl/repository/freearticles/597493.pdf.
7. Matsumoto, M. et Nishimura, T. (1998). Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudo-Random Number Generator. ACM Transactions on Modeling and Computer Simulation, vol. 8 (1), p. 3-30.
8. Marsaglia, G. (1968). Random Numbers Fall Mainly in the Planes. Proceedings of the National Academy of Sciences of the United States of America, vol. 61 (1), p. 25-28.
9. Universität Salzburg. (s.d.). Collection of Pseudorandom Number Generators [en ligne], https://web.archive.org/web/20181118052935if_/http://random.mat.sbg.ac.at/results/karl/server/server.html.
10. Diaconis, P. et Graham, R. (2012). Magical Mathematics: The Mathematical Ideas That Animate Great Magic Tricks. Princeton : Princeton University Press.
11. Berlekamp, E. R. (1967). Factoring Polynomials Over Finite Fields. Bell System Technical Journal, vol. 46 (8), p. 1853-1859.
12. Cantor, D. G. et Zassenhaus, H. (1981). A New Algorithm for Factoring Polynomials over Finite Fields. Mathematics of Computation, vol. 36 (154), p. 587-592.